Atividade de registro semana 26 de versionamento
Nome: Arthur Saraiva de França 3b.

Parte 1:

Etapa 1:

-Quais serviços são necessários para que o ambiente funcione?

O broker Kafka e a aplicação Kafka Streams.

-Qual serviço consome e processa os eventos?

A aplicação Kafka Streams.

-Qual serviço representa a infraestrutura de mensageria?

O broker Kafka.

Etapa 2:

-Por que empacotar a aplicação ajuda a repetir o deploy com mais consistência?

Porque o empacotamento em container inclui o código, as dependências e as configurações necessárias dentro de uma imagem isolada. Isso garante que a aplicação vai rodar exatamente do mesmo jeito em qualquer computador, sem erros causados por falta de programas ou versões diferentes instaladas na máquina.

Etapa 3:

-Qual serviço representa the broker?

O serviço kafka.

-Qual serviço representa a aplicação Streams?

O serviço app-streams.

-Por que o app depende do Kafka?

Porque a aplicação Streams precisa que o broker Kafka já esteja iniciado e pronto para funcionar para conseguir se conectar, ler e processar os tópicos de mensagens.

Etapa 4:

-Qual situação parece normal?

A situação com o lag baixo e estável.

-Qual situação exigiria investigação da equipe?

A situação com o lag alto e aumentando continuamente.

-O que a equipe poderia fazer se o lag continuar aumentando por muito tempo?

A equipe poderia realizar uma escala horizontal, aumentando a quantidade de instâncias da aplicação rodando em paralelo para processar as mensagens mais rápido e diminuir o acúmulo na fila.

Etapa 5:

Registro:

O ambiente analisado é composto pelo broker Kafka e pela aplicação Kafka Streams. O Docker Compose ajuda a organizar o deploy ao centralizar a configuração de múltiplos containers em um único arquivo, iniciando o ambiente e conectando os serviços de forma automática. O consumer lag deve gerar atenção operacional quando estiver alto e crescendo continuamente, sinalizando que a aplicação está lenta e não consegue acompanhar o volume de mensagens recebidas.

Parte 2:

Etapa 1:

-Quais eventos pertencem ao mesmo pedido?

Os eventos que compartilham o mesmo identificador, sendo o pedido 101 (criado às 10:00 e pago às 10:04) e o pedido 102 (criado às 10:03 e pago às 10:10).

-Quais pagamentos aconteceram até 5 minutos após a criação do pedido?

Apenas o pagamento do pedido 101, pois a diferença entre a criação (10:00) e o pagamento (10:04) foi de 4 minutos. O pedido 102 levou 7 minutos e passou do limite.

Etapa 2:

-Por que a mesma chave, sozinha, não garante o join?

Porque em fluxos de dados contínuos (streams), os eventos precisam estar relacionados também no tempo. Se não houver um limite temporal (janela), o

sistema ficaria esperando um pagamento indefinidamente, guardando dados antigos na memória e misturando transações de períodos totalmente diferentes.

Etapa 5:

Registro:

A janela de 5 minutos funciona como uma regra de corte temporal para a operação de join. Ela determina que dois eventos com a mesma chave só serão combinados se o segundo acontecer dentro desse intervalo máximo após o primeiro. Isso permite filtrar e correlacionar apenas as ações que fazem sentido lógico para o negócio em tempo real, descartando automaticamente processos que estouraram o tempo limite de atendimento.